# MAT 586
# Lab 2

**Nolan R. H. Gagnon**

**Ex. 5.6**

In the figure below, we plot the difference between final population sizes for logistic models with different initial conditions. In particular, for each power $p$ on the x-axis, we plot the difference between the final population of a logistic model with initial condition $x_0 = 0.2$ and the final population of a logistic model with initial condition $x_0 = 0.2 + 10^{-p}$.

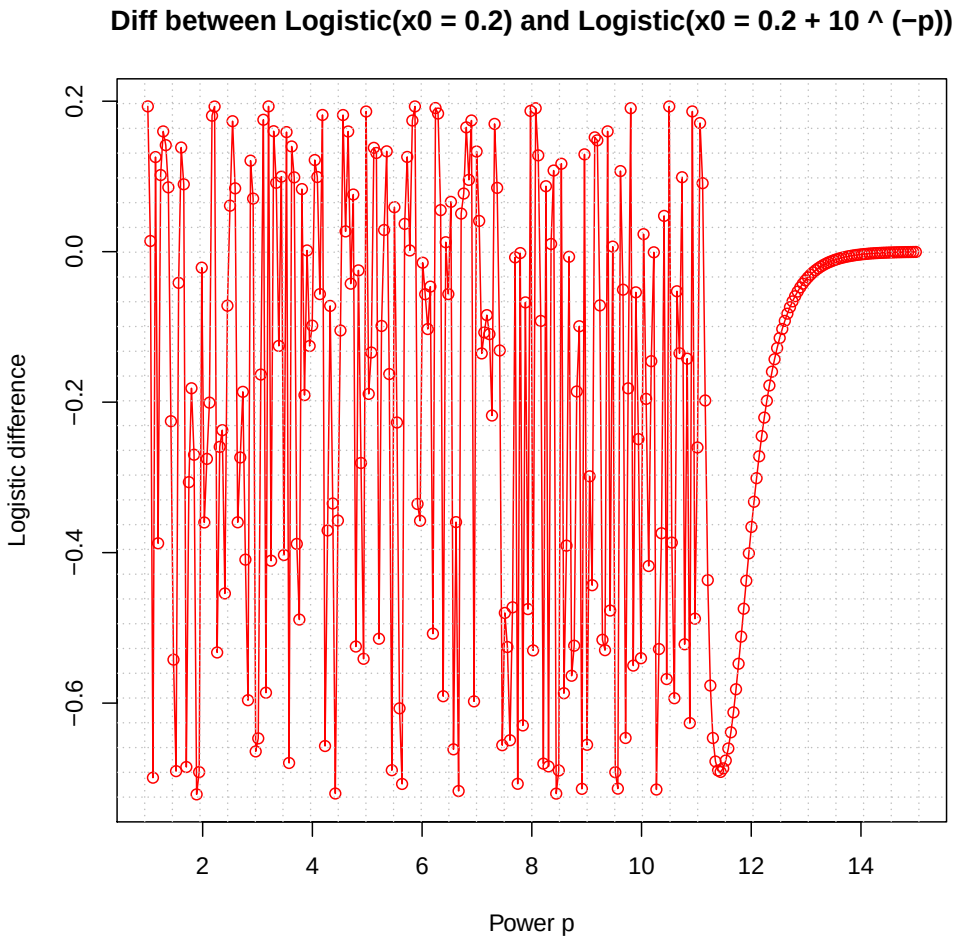


Figure 1: Logistic Difference

The R code that produced the plot on the previous page is provided below. Note that the source code for **lab2logistic.R** is provided in the Appendix.

### R Code

```
#####################################################################
## FILENAME: exercise_ch5_no6.R                                    ##
## PURPOSE: Compares results of logistic model simulations         ##
##          with different initial conditions.                     ##
## AUTHOR: nolanHg                                                 ##
## DATE: 05/12/2018                                                ##
#####################################################################

# provides access to the lab2logistic() function, which
# runs logistic model simulations
source("lab2logistic.R")

##
## BRIEF: Takes a vector of powers p and runs a logistic model simulation
##        with initial condition x0 + 10 ^ (-p) for each p. Then the
##        difference between the final population of logistic(x0 + 10 ^ (-p))
##        and the final population of logistic(x0) is plotted for each p.
##
## PARAM power.v: A vector of powers p that are used in the logistic model
##                initial conditions.
##
## PARAM x0: Initial condition for the model logistic(x0).
##
## PARAM numSteps: Number of discrete time steps to use in logistic model
##                 simulations.
##
## PARAM lambda0: Per-capita rate.
##
## RETURNS: Void.
logisticDifference <- function(power.v = seq(1, 15, len = 300), x0 = 0.2, numSteps = 50, lambda0 =
    3.93)
{
   # run logisitic model using initial condition x0
   temp <- lab2logistic(lambda0 = lambda0, numSteps = numSteps, x0 = x0, plotType = 0)

   # get population size vector from logistic model simulation
```

```
  x <- temp[[2]]

  # get the final population size and store it in result1
  result1 <- tail(x, 1)

  # create a vector to store results of logistic model simulation with initial conditions x0 + 10 ^
      (-p)
  # for each p in power.v
  results2.v <- rep(NA, length(power.v)) #

  # for each p in power.v, run logistic model simulation with initial condition x0 + 10 ^ (-p)
  # and store final population size in results2.v
  for (i in 1 : length(results2.v)) {

    p <- power.v[i]
    temp <- lab2logistic(lambda0 = lambda0, numSteps = numSteps, x0 = x0 + 10 ^ (-p), plotType = 0)
    x <- temp[[2]]
    results2.v[i] <- tail(x, 1)

  }

  # compute the differences in the final populations for logistic(x0) and logistic(x0 + 10 ^ (-p))
  # for each p in power.v
  logistic_diff.v <- results2.v - result1

  # plot the logistic differences versus p
  plot(power.v, logistic_diff.v, type = "o", xlab = "Power p", ylab = "Logistic difference",
       main = sprintf("Diff between Logistic(x0 = %g) and Logistic(x0 = %g + 10 ^ (-p))", x0, x0),
       col = "red")

  # add gridlines to the plot
  grid(nx = 30, ny = 30, col = "gray", lty = "dotted")
}
```

■

**Ex. 5.7**

The R code for **lab2predPreyEquil()** is provided below.

```
######################################################################
## FILENAME: exercise_ch5_no7.R                                     ##
## PURPOSE: Computes the equilibrium values for the                 ##
##         predator-prey model.                                     ##
## AUTHOR: nolanHg                                                  ##
## DATE: 05/12/2018                                                 ##
######################################################################

## BRIEF: Computes the equilibrium values for the predator-prey model.
##
## PARAM lambda0: Prey's per-capita rate when populations of prey and predators
##                are small.
##
## PARAM psi: First decay parameter for prey's per-capita rate.
##
## PARAM alpha: Second decay parameter for prey's per-capita rate.
##
## PARAM gamma: Per-capita rate for predators when prey are scarce.
##
## PARAM beta: Asymptotic per-capita rate for predators when prey
##             are abundant.
##
## PARAM theta: Parameter that controls how quickly increasing prey
##              density saturates the per-capita predator growth rate.
##
## RETURNS: Vector containing equilibirum values for prey and predator populations.
lab2predPreyEquil <- function(lambda0 = 2.5, psi = 1, alpha = 0.3, gamma = 0.5, beta = 2, theta = 1)
{
  # compute the equilibrium value for the prey population
  v_eq <- (1 / theta) * log((beta - gamma) / (beta - 1))

  # compute the equilibrium value for the predator population
  p_eq <- (1 / alpha) * ((log(lambda0) / v_eq) - psi)

  # put both equilibrium values into a vector
  eq.v <- c(v_eq, p_eq)
```

```
    # return vector of equilibrium values to the caller
    return(invisible(eq.v))
}
```

In the screen-shot below, an R session in which the equilibrium population sizes are calculated is shown. The equilibrium values shown on line 5 of the screenshot were calculated using the R function in the code listing above. The equilibrium values shown on line 9 were calculated by simply taking the last prey and predator population sizes from the output of the predator-prey simulation code provided in the course textbook (Hiebeler § 5.4.2).

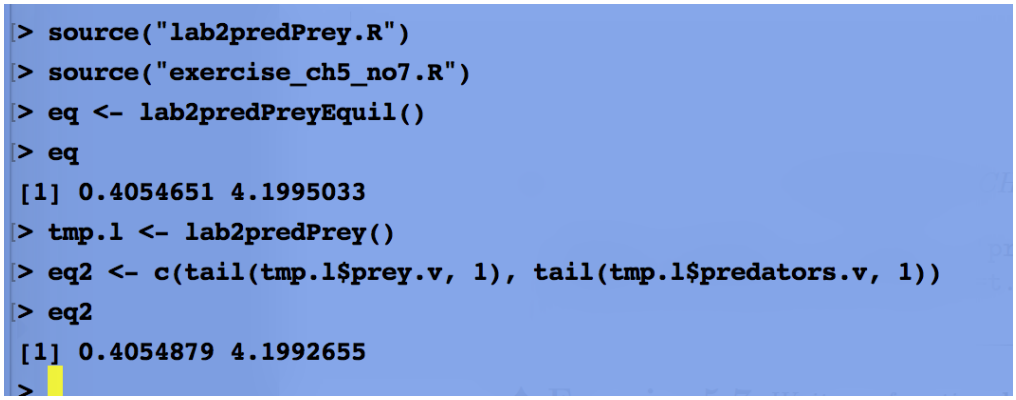


Figure 2: Finding Equilibria

As the screen-shot shows, the two sets of equilibrium values are the same up to 4 decimal places for the prey populations and up to 3 decimal places for the predator populations. Note that the source code for **lab2predPrey.R** is included in the Appendix.

■

**Ex. 5.8**

The screen-shot below shows an R session in which the predator-prey simulation was run with $\lambda_0 = 1.4$.

```
> source("lab2predPrey.R")
> source("exercise_ch5_no7.R")
> eq <- lab2predPreyEquil(lambda0 = 1.4)
> eq
[1]  0.4054651 -0.5671912
> tmp.l <- lab2predPrey(lambda0 = 1.4)
> eq2 <- c(tail(tmp.l$prey.v, 1), tail(tmp.l$predators.v, 1))
> eq2
[1] 3.364059e-01 1.619765e-08
```

Figure 3: Breakdown of Equilibrium Formulae

On line 5, the equilibrium solutions given by

$$v^* = \frac{1}{\theta}\log\left(\frac{\beta-\gamma}{\beta-1}\right), \quad p^* = \frac{1}{\alpha}\left(\frac{\log(\lambda_0)}{v^*} - \psi\right)$$

are shown. On line 9, the last population sizes for the predators and prey are shown. Clearly, the two sets of values do not agree. In fact, the equilibrium solutions yields a predator population size that doesn't make sense biologically (it is negative). This can be explained mathematically. By default, the R functions **lab2predPrey()** and **lab2predPreyEquil()** use $\theta = 1, \psi = 1, \alpha = 0.3, \gamma = 0.5$, and $\beta = 2$. This means

$$v^* = \frac{1}{\theta}\log\left(\frac{\beta-\gamma}{\beta-1}\right) = \log(1.5).$$

Now, notice that if we choose $\lambda_0 = 1.5$, we will have

$$p^* = \frac{1}{\alpha}\left(\frac{\log(1.5)}{v^*} - \psi\right) = \frac{1}{0.3}\left(\frac{\log(1.5)}{\log(1.5)} - 1\right) = 0.$$

If we choose $0 < \lambda_0 < 1.5$, we will have $p^* < 0$. So for $\lambda_0$ such that $\log(\lambda_0) < v^*$, our equilibrium solutions break down.

■

**Appendix**: Source code included in R solutions

**lab2logistic.R** Author: Prof. David Hiebeler

```
## Function: lab2logistic
## Perform a simulation of discrete-time logistic model and
## plot the results.
## Inputs:
## numSteps: the number of time steps to run
## x0: the initial condition
## lambda0: the intrinsic growth parameter
## plotType: a variable indicating the type of plot to produce:
## 1 means make a new plot of this simulation
## 2 means add a plot of this simulation to an existing plot
## any other value means don't produce a plot
## ...: any additional arguments to pass to the "plot" command
## Outputs:
## (invisibly) A list containing two vectors: time.v and popSize.v
## A plot is also optionally produced
lab2logistic=function(numSteps=50, x0=0.23, lambda0=1.9, plotType=1, ...) {
t.v = rep(NA,numSteps+1); x.v = rep(NA,numSteps+1) # allocate space
t.v[1] = 0; x.v[1] = x0 # initial conditions
for (i in 1:numSteps) {
t.v[i+1] = t.v[i] + 1
x.v[i+1] = lambda0 * x.v[i] * (1 - x.v[i])
}
if (plotType == 1) {
## plot results, specifying the range for y coordinates
plot(t.v, x.v, type='o', ylim=c(0,1), ...)
} else if (plotType == 2) {
lines(t.v, x.v, type='o', ...) # add to existing plot
}
return(invisible(list(time.v=t.v, popSize.v=x.v)))
}
```

**lab2predPrey.R** Author: Prof. David Hiebeler

```
## Function: lab2predPrey
## Simple predator-prey model
## Inputs:
## numSteps: the number of time steps to run
## v0: initial number of prey
## p0: initial number of predators
## Outputs:
## (invisibly) A list of three vectors: time.v, prey.v, predators.v
## A plot is also produced
lab2predPrey = function(numSteps=50, v0=5, p0=5, lambda0=2.5, psi=1,
alpha=0.3, gamma=0.5, beta=2, theta=1) {
## initialize t.v, v.v, and p.v
t.v = rep(NA, numSteps+1)
v.v = rep(NA, numSteps+1); p.v = rep(NA, numSteps+1)
t.v[1] = 0
v.v[1] = v0; p.v[1] = p0
## main loop
for (k in 1:numSteps) {
t.v[k+1] = t.v[k] + 1
v.v[k+1] = lambda0*v.v[k]*exp(-psi*v.v[k] - alpha*v.v[k]*p.v[k])
p.v[k+1] = p.v[k]*(beta - (beta-gamma)*exp(-theta*v.v[k]))
}
## plot results
matplot(t.v, cbind(v.v, p.v), type='l', col='black', lty=c(1,2))
legend('topleft', c('prey', 'predator'), lty=c(1,2))
return(invisible(list(time.v=t.v, prey.v=v.v, predators.v=p.v)))
}
```